COHERE LABS · ML MATH · 2 NOVEMBER 2026

When the Update Is Not a Gradient

Input-to-state stability of self-repairing neural network training

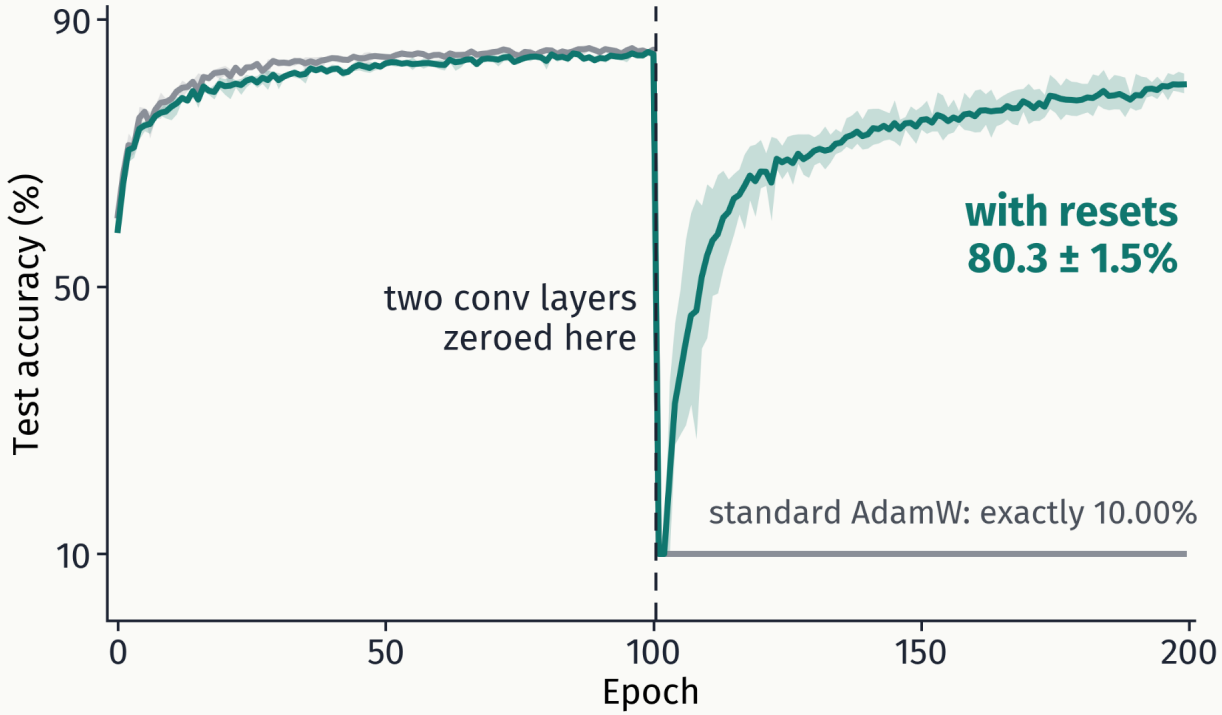
Rokib Al Dhin Raadh

Independent researcher, Dhaka · poster at DeepMath 2026, The Ohio State University

raadh.me/deepmath

A PUZZLE

Zero two layers, and gradient descent never recovers



10.00%

forever, with standard AdamW (3 of 3 seeds)

80.3%

after a controller resets the dead layers

Why is gradient descent stuck? And is it safe to let a controller jump in?

WHY REPAIR AT ALL

Long training kills units, so people repair them mid-training

Train long enough, or on data that keeps changing, and units die: they stop firing and stop learning. This is loss of plasticity (Dohare et al., *Nature* 2024).

Repair method	Paper	What it does mid-training
Continual Backprop	Dohare et al., Nature 2024	re-draws the least useful units
ReDo	Sokar et al., ICML 2023	re-draws dormant neurons
Shrink & Perturb	Ash & Adams, NeurIPS 2020	shrinks the weights, adds fresh noise
Plasticity injection	Nikishin et al., NeurIPS 2023	adds fresh trainable parameters

None of these updates is the gradient of any loss.

THE PROBLEM

A repair is not a gradient step, so the usual proofs don’t apply

$$\theta_{t+1} = \theta_t - \eta \nabla L(\theta_t) + u_t$$

Gradient descent plus an extra term u, the repair. Anything that isn’t the gradient goes into u.

Triggered

It fires only when a diagnostic crosses a threshold.

State-dependent

What it does depends on the current weights.

Discontinuous

A reset jumps. No loss has it as its gradient.

When does a network that repairs itself still converge?

WHERE THE IDEA CAME FROM

Control theory has asked my question since 1868

When does a system that keeps getting pushed still stay on course?

1868

Maxwell

Why do steam-engine governors start to shake?

1892

Lyapunov

Prove a motion is stable without solving its equations.

1950s–60s

Rockets and autopilots

The wind keeps pushing. Stay on course anyway.

1989

Sontag

Input-to-state stability: a bounded push gives a bounded drift.

1999

Hespanha & Morse

A system that switches modes: how often is too often?

2026

A neural network

Training is the flight. Repairs are the gusts.

So I asked: why not a neural network?

THE IDEA

Training with repairs is a rocket in the wind

A rocket in flight	A network in training
The flight path	Gradient descent (flow)
Gusts of wind	Small repairs (flicks)
A stage separation	A reset (a jump)
Staying inside the flight corridor	Staying near a good solution
“Will it stay on course?”	“Will training still converge?”

Control theory has a name for “stays on course while being pushed”: input-to-state stable.

THE ANSWER, IN ONE SLIDE

Small repairs keep training stable. Only a jump can revive a dead layer.

Both are proved. Together, training is smooth flow, small flicks, and the occasional jump.

Part 1 · Proposition 1

Part 2 · Theorem 2

PART 1 OF 2

Small repairs are safe

The system, the control-theory tool, and a prediction we can test

THE SETUP

A network that repairs itself while it trains

Flow

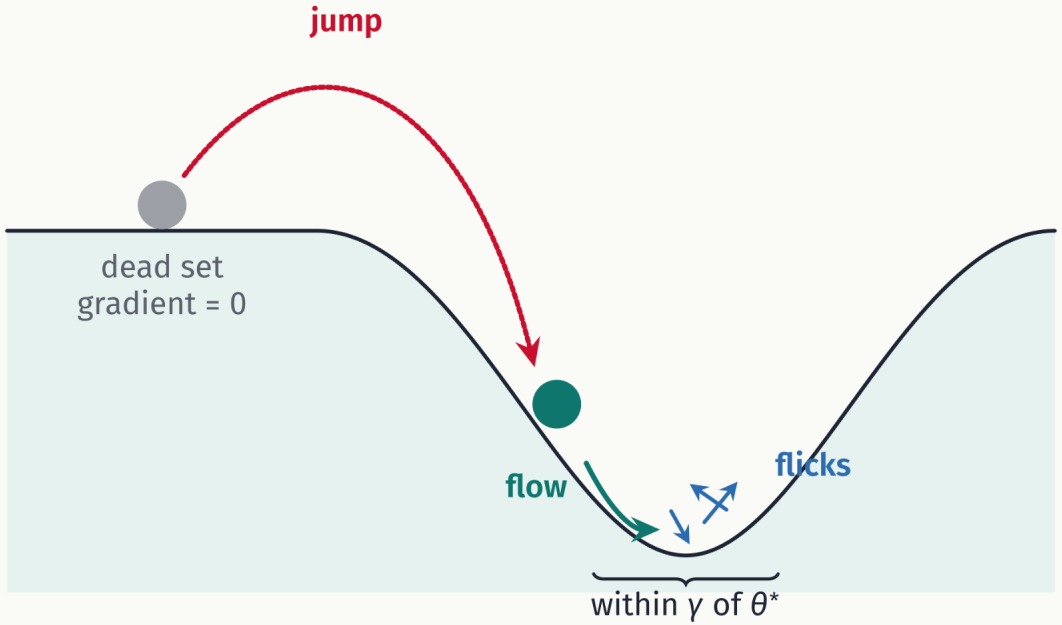
Normal gradient steps on the main loss, plus a small local loss for each layer, with weight β.

Flicks

Each layer has a health score. When it drops sharply, a small correction fires, always capped.

Jumps

A layer dead for three epochs in a row is re-drawn from scratch.



Neither result needs this exact controller. Part 1 needs only bounded repairs; Part 2 needs only Conv–BN–ReLU, AdamW and repairs bounded per channel.

THE TOOL FROM CONTROL THEORY

ISS: a bounded push gives a bounded drift

A system driven by an input u is **input-to-state stable** if, for every start and every input,

$$\|x_t\| \le \kappa(\|x_0\|,\ t) + g(\sup \|u_k\|) + c, \text{ sup over } k < t$$

κ: where you started

Fades to zero over time. The system forgets its start.

g: how hard it was pushed

Grows with the biggest push so far. No push, no effect.

c: a small floor

c = 0 is ISS (Sontag 1989). c > 0 is practical ISS (Jiang, Teel & Praly 1994).

Discrete-time ISS: Jiang & Wang 2001.

ISS doesn’t care what the input is. It only has to be bounded.

THE SIMPLEST EXAMPLE

A ball in a bowl is already ISS

Each step pulls θ back toward the bottom. A repair can push it by up to ε:

$$L(\theta) = (\alpha/2)\,\theta^2,\ \theta_{t+1} = (1 - \eta\alpha)\,\theta_t + u_t,\ |u_t| \le \varepsilon$$

The pull-back and the pushes balance out:

$$|\theta_t| \le (1 - \eta\alpha)^t\,|\theta_0| + \varepsilon\ /\ (\eta\alpha)$$

That's the ISS estimate, with $\kappa(r,\ t) = (1 - \eta\alpha)^t\ r,\ g(s) = s\ /\ (\eta\alpha),\ c = 0$

Proposition 1 is this, in many dimensions.

PROPOSITION 1 · PRACTICAL ISS

Bounded repair is safe

Near a minimum θ*, let the loss be ℓ-smooth and α-strongly convex, with step size η at most 1/ℓ. Training follows

$$\theta_{t+1} = \theta_t - \eta \nabla L(\theta_t) - \eta\, d_t + u_t,\ \|d_t\| \le 2\beta M,\ \|u_t\| \le \varepsilon_t$$

d is the local-loss term from the setup (weight β, with its projection); u is the repair. While θ stays near θ*,

$$\limsup \|\theta_t - \theta^*\| \le \gamma = 2\beta M/\alpha + \bar{\varepsilon}/(\eta\alpha)$$

As t goes to infinity. $\bar{\varepsilon}$ is the long-run size of the repairs (their limsup).

Repairs push training around, but never further than γ from a good solution.

PROPOSITION 1 · HOW TO READ IT

What the bound says, and what it doesn’t

A floor from the local loss

$2\beta M/\alpha$ stays even with no repairs, so this is practical ISS. Fade β to zero and the floor goes.

Repairs aren’t scaled by η

A repair pushes by ε whatever the step size. Smaller steps: weaker pull-back, same push.

It makes a prediction

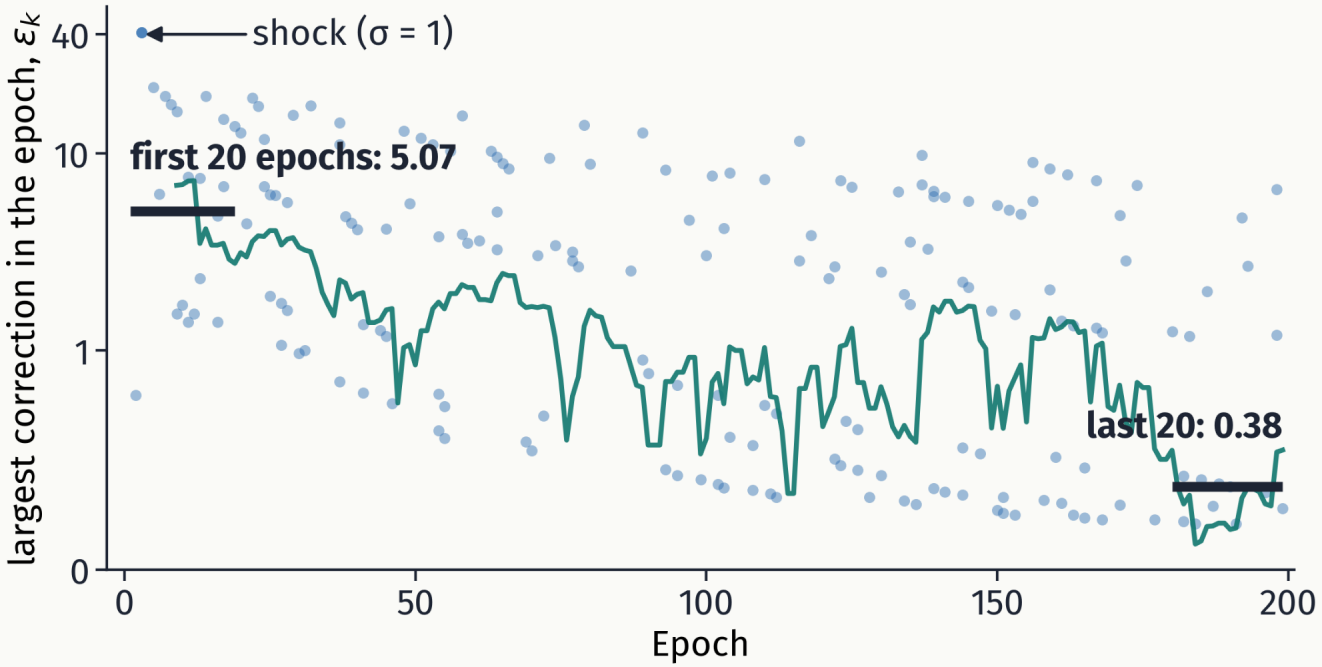
γ is small only if the repairs shrink over time. That’s testable.

Loose, and local

With our settings γ comes out at 3,800, so read it for its shape, not its size. It holds only near θ^*, and BatchNorm breaks strong convexity.

TESTING THE PREDICTION

After a shock, the repairs shrink by 93%



−93%

Largest correction per epoch: **5.07** over the first 20 epochs, **0.38** over the last 20.

CIFAR-10 CNN. Shock: Gaussian noise, σ = 1, added to every backbone weight at epoch 2. Three seeds. Blue: each epoch's largest correction. Teal: 10-epoch average.

PART 2 OF 2

Only a jump can revive a dead layer

Theorem 2, its proof, and the puzzle from the start

THEOREM 2 · ABSORBING DEAD SET

Bounded repair cannot revive

In a Conv–BN–ReLU layer with BatchNorm in train mode, take one channel c, with filter W and BatchNorm shift b. Its dead set is

$$D = \{\ W = 0,\ b \le 0\ \},\ \theta_0 \in D \text{ implies } \theta_t \in D \text{ for all } t$$

D is invariant under AdamW with fresh state, and under every repair that is bounded per channel:

$$\|R(W) - W\| \le f\,\|W\|$$

Gradient descent and small fixes are both stuck, exactly. Only a jump leaves D.

s and b are BatchNorm's per-channel scale and shift, often written γ and β.

THEOREM 2 · PROOF

Four exact steps

1 · A zero filter feeds BatchNorm a constant (a bias is a constant too), which it normalises to zero.

$$\hat{x} = (x - \text{batch mean}) \,/\, \sqrt{(\text{batch variance} + \epsilon)} = 0$$

2 · The channel outputs ReLU(b) = 0, and the ReLU is closed there.

$$y = s\,\hat{x} + b = b \le 0, \text{ so } \mathrm{ReLU}(y) = 0 \text{ and } \mathrm{ReLU}'(y) = 0$$

3 · So every gradient into the channel is exactly zero.

$$\partial L/\partial W = \partial L/\partial s = \partial L/\partial b = 0$$

4 · AdamW with fresh state can't move it, and neither can a bounded repair.

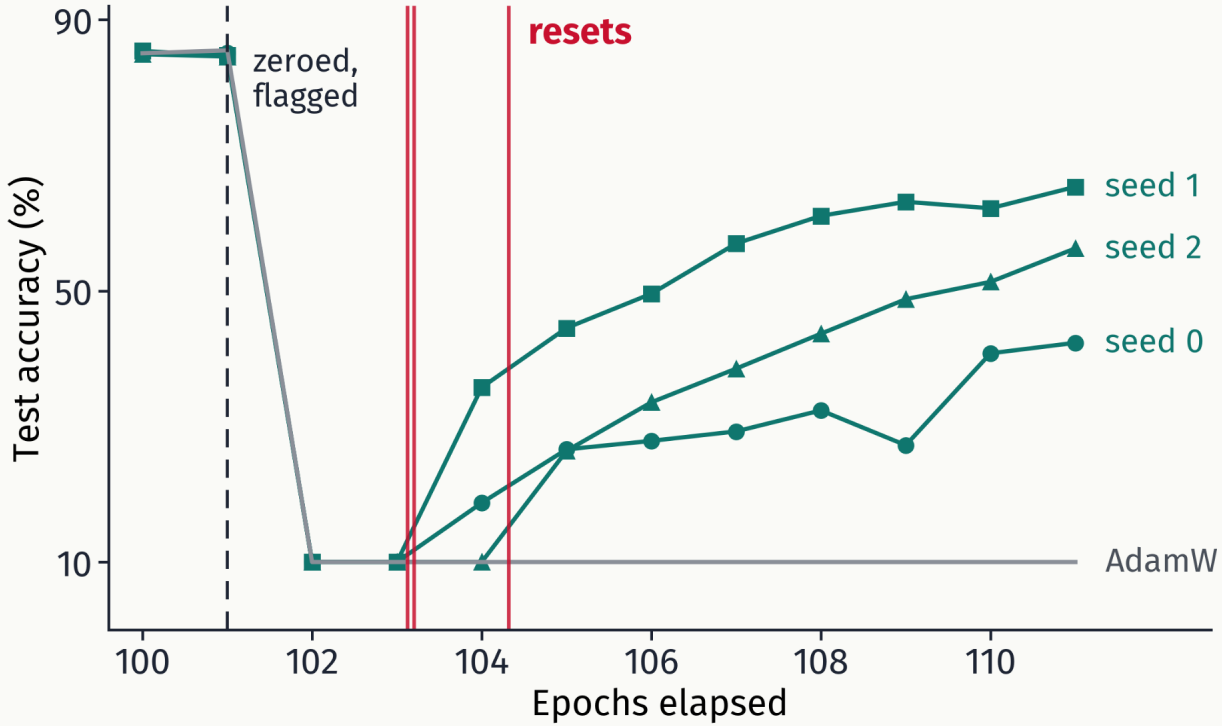
$$m = v = 0, \text{ so new } W = (1 - \eta\lambda)\; W = 0 \text{ and} b_t = (1 - \eta\lambda)^t\; b_0 \le 0$$

$$\|R(W) - W\| \le f\,\|W\| = 0, \text{ so } R(W) = W = 0$$

By induction, θ stays in D forever. A relative bound can't move zero; only an additive jump can.

BACK TO THE PUZZLE

In every seed, recovery starts exactly at the reset



10.00%

standard AdamW, all 99 epochs after the lesion, 3 of 3 seeds

Step 39 493

dead layers flagged, 2 steps after the lesion, in every seed

Epochs 103–104

resets fire; each seed rises only after its own reset

80.3 ± 1.5%

final accuracy with resets (85% before the lesion)

WHAT THEOREM 2 EXPLAINS

Each of these fixes contains a jump

Method	Bounded part (flick)	Jump part (can leave D)
Continual Backprop	none	re-draws low-utility units
ReDo	none	re-draws dormant neurons' inputs
Shrink & Perturb	shrinks weights toward zero	adds fresh noise
Plasticity injection	none	adds fresh trainable parameters

A correction bounded relative to the weights can't move a zero filter, so a method that only rescales or clips can never revive a dead channel. **The jump isn't a design accident. It's necessary.**

L2 Init (Kumar, Marklund & Van Roy, CoLLAs 2024) escapes differently: it pulls weights toward their initial values, which changes the loss, so D is no longer invariant.

WHERE REPAIR STOPS MATTERING

With a skip path, there is nothing to repair

92.3%
without repair (± 0.3)

91.6%
with repair (± 0.4)

ResNet-18 on CIFAR-10, the same lesion on convolutions behind a skip connection, three-seed means. The skip path carries the signal past the zeroed layers.

Repair matters when there’s no redundancy to fall back on.

OPEN PROBLEMS

How often may a network jump?

Proposition 1 holds between jumps; Theorem 2 says jumps are unavoidable. Find a dwell time such that

$$\text{jumps at least } \tau \text{ steps apart implies limsup } r_t \le \gamma'$$

cf. average dwell time for switched systems, Hespanha & Morse 1999

PL, not strong convexity

BatchNorm's scale invariance rules out strong convexity. Cui et al. (2025) have ISS under generalised PL. Do BN nets satisfy it?

Derive the repair sizes

We measure how the repairs shrink. Derive that decay from the repair rule itself.

Optimiser state

Theorem 2 needs fresh AdamW state. Adam's moments belong in the state of the ISS analysis.

FROM THEOREM TO TOOL

Theorem 2’s conditions are exact, so you can check them

```
pip install ormas-monitor
```

Certifies

Checks the conditions during PyTorch training, certifies dead channels, and can repair them.

Cheap

+3.6% epoch time on our CIFAR-10 CNN, on one Mac. Large models not measured yet.

Real models

Near-zero channels in 19 of 21 torchvision BN–ReLU models. Not exactly zero, so not certified.

Removing VGG-19-BN’s 292 near-dead channels cut compute by **18.6%**, with identical top-1 answers on 64 test images.

CORRECTIONS

Corrections to the abstract and the poster

Earlier claim	Corrected
Correction size fell 83–93% (abstract)	93% for the largest correction per epoch after a shock, 3 seeds; 90% counting only corrections that fired. Not to zero.
Contraction holds on 31.6% of steps (abstract)	Not a contraction measurement. It's the share of steps where $\varepsilon/(\eta\alpha) < 1$, essentially steps with no correction, in one seed; the other seeds give 71% and 76%. Contraction itself isn't measured yet.
ResNet-18: 92.6% vs 91.7% (abstract)	Those were seed 0. Three-seed means: 92.3% vs 91.6%.
First reset at step 39 493 (poster)	That's when the dead layers are flagged. The resets fire three epochs later, at epochs 103–104.

The lesion and shock numbers in this talk were recomputed from the raw run logs.

WHAT I SHOWED

1 · Treat a repair as an input, and bounded repairs are provably safe. The bound predicts they must shrink, and they do: −93%.

2 · Bounded repair can never revive a dead channel. That's why every working fix contains a jump.

3 · Open: how often may a network jump?

Challenge the bound.

raadh.me/deepmath · pip install ormas-monitor · raadh@oxiedo.com

Three literatures meet here

Control theory

- Sontag 1989 · input-to-state stability
- Jiang, Teel & Praly 1994 · practical ISS
- Jiang & Wang 2001 · discrete-time ISS
- Hespanha & Morse 1999 · average dwell time
- Liberzon 2003 · switching in systems and control

Optimisation

- Devolder, Glineur & Nesterov 2014 · inexact first-order oracles
- Cui, Jiang, Sontag & Braatz 2025 · ISS of perturbed gradient descent
- Arora, Li & Lyu 2019 · BatchNorm and scale invariance
- Yu et al. 2020 · gradient projection (PCGrad)

Plasticity

- Dohare et al. 2024 · loss of plasticity
- Sokar et al. 2023 · dormant neurons, ReDo
- Kumar, Marklund & Van Roy 2024 · L2 Init
- Hernandez-Garcia et al. 2025 · re-draw weights or units?
- Han et al. 2026 · FIRE: where to re-initialise
- He & Liu 2025 · silent neurons, forward and backward

PROPOSITION 1 · PROOF

The proof is the parabola argument

1 · The gradient step contracts $\|\theta - \eta\nabla L(\theta) - \theta^*\| \le \rho\,\|\theta - \theta^*\|,\ \rho = 1 - \eta\alpha,\ \text{for } \eta \le 1/\ell$

2 · Triangle inequality on d and u $r_{t+1} \le \rho\, r_t + 2\eta\beta M + \varepsilon_t,\ \text{where } r_t = \|\theta_t - \theta^*\|$

3 · Unroll the recursion $r_t \le \rho^t\, r_0 + \Sigma_k\, \rho^{t-1-k}\,(2\eta\beta M + \varepsilon_k),\ \text{sum over } k < t$

4 · Sum the geometric series $\text{limsup}\ r_t \le (2\eta\beta M + \bar{\varepsilon})\ /\ (1 - \rho) = 2\beta M/\alpha + \bar{\varepsilon}/(\eta\alpha)$

This is the standard analysis of gradient descent with errors (cf. Devolder, Glineur & Nesterov 2014; for ISS of perturbed descent, Cui, Jiang, Sontag & Braatz 2025). What's new is the input we feed it: the repair.

TESTING THE PREDICTION · A CLOSER LOOK

Smaller, not just rarer. But not zero.

Measure, 3 seeds	First 20 epochs	Last 20 epochs	Change
Largest correction per epoch, mean	5.07	0.38	−93%
Size, given a correction fired	13.1	1.34	−90%
Epochs with any correction	39%	28%	−11 points
Largest single correction	41.0	6.57	−84%

The repairs really do shrink: a late correction is about 90% smaller. But late corrections still reach 6.6, so $\bar{\varepsilon} > 0$. **The repair term shrinks; it doesn't vanish. That's practical stability, not convergence to θ^*.**

Isn’t Theorem 2 just the dead-ReLU problem?

The mechanism is simple, and I say so. What the theorem adds:

- Exact conditions: train-mode BatchNorm, fresh optimiser state, non-positive shift.
- Invariance under every per-channel-bounded repair, not only gradient steps.
- That makes it checkable: ormas-monitor certifies exactly this set.
- It’s the lower bound behind the switched-system picture: some repairs must be jumps.

You zeroed the layers yourself. Does this happen in real models?

The exact zeros in the lesion are by construction. In released models, the tool finds near-zeros:

- 19 of 21 torchvision BatchNorm–ReLU models have channels whose filter or BatchNorm scale is below 0.00001 of the layer median, with shift at or below 0.
- They output exactly 0 on real images in ResNet-18, VGG-19-BN and MnasNet, and at most 0.0000015 in DenseNet-169.
- They aren’t exactly zero, so Theorem 2 doesn’t certify them.
- Removing VGG-19-BN’s 292: 18.6% less compute, identical top-1 answers on 64 images.

Shrinking BatchNorm scales is a known pruning technique (network slimming, Liu et al. 2017). The point is that one command finds them.

With no repair, γ is still 2βM/α. Why doesn't training converge to θ*?

Because training also follows each layer's local loss, with weight β of at most 0.3, which pulls against the global loss.

The local-loss gradient and its projection add up to 2ηβM per step, whether or not a repair fires. That offset is why Proposition 1 is **practical** ISS.

It becomes strict ISS only if β(t) goes to 0, for example by annealing the local-loss weight.

What keeps θ near θ* in Proposition 1?

Induction. If the worst-case radius fits inside the strongly convex ball of radius R, the iterates never leave it:

$$r_0 \le R \text{ and } 2\beta M/\alpha + \varepsilon_{\max}/(\eta\alpha) \le R \text{ implies } r_t \le R \text{ for all } t$$

Each step gives r at most (1 − ηα)R + ηαR = R, so the ball is invariant.

So the statement is self-consistent. With realistic constants it isn't checkable, which is why I call it idealised.

Why does the radius grow as the step size shrinks?

$\eta\alpha \cdot r$ (pull-back per step) $= \varepsilon$ (push per repair), so $r = \varepsilon \,/\, (\eta\alpha)$

A repair is an absolute move of size ε; it isn't multiplied by η. Gradient descent pulls back only about ηα of the distance per step. Smaller η: slower pull-back, same push.

The fix: scale the repairs with η, or anneal them with the learning rate.

In our runs the repairs also shrink late in training (about 90% smaller when one fires), which offsets part of this.

How does this relate to Cui, Jiang, Sontag & Braatz (2025)?

Their result

Perturbed gradient descent is small-disturbance input-to-state stable, under a generalised Polyak–Łojasiewicz condition. The disturbance is generic.

What this work adds

The disturbance is a repair: triggered, state-dependent, discontinuous. The measurable condition the bound needs (repairs must shrink). And Theorem 2: bounded repair alone cannot leave a dead set.

Their analysis is the right engine for a PL version of Proposition 1.

Strong convexity can’t hold with BatchNorm. Doesn’t that break Proposition 1?

It makes Proposition 1 a local, idealised statement. BatchNorm makes the loss invariant to rescaling a layer’s incoming weights (Arora, Li & Lyu 2019), so it’s flat along those directions: no strong convexity.

The natural replacement is a PL-type condition. Cui et al. (2025) already have ISS under generalised PL. Whether BatchNorm networks satisfy it near the minima we care about is the open step.

Theorem 2 doesn’t use convexity at all.

Does Theorem 2 depend on the optimiser state? And why exactly 10.00%?

Optimiser state: yes

Leftover momentum could move a zero filter, so the theorem assumes fresh AdamW state, and our harness clears it at the lesion. With zero gradients, weight decay shrinks the shift b by $(1 - \eta\lambda)$ each step; the logs match to the bit.

Exactly 10.00%

With the layers zeroed, the output no longer depends on the input, so the network predicts one class for every image. CIFAR-10's test set has exactly 1,000 images per class.

Edge case: in the monitor's full demo run every channel of both zeroed layers met the conditions. In its quick run, 2 of 64 channels had a positive shift; those output a constant and are reported as constant, not certified dead.

Does the jump do all the work?

For reviving, yes: that’s Theorem 2. In all three seeds, accuracy stays at exactly chance until the reset’s epoch, and rises from that epoch on.

The small corrections then help finish the recovery. Whether they beat plain resets is an open empirical question.

Next experiment: reset-only and ReDo baselines over several seeds, with the same detector.

References

- Arora, Li & Lyu. Theoretical analysis of auto rate-tuning by batch normalization. ICLR 2019.
- Ash & Adams. On warm-starting neural network training. NeurIPS 2020.
- Cui, Jiang, Sontag & Braatz. Perturbed gradient descent algorithms are small-disturbance input-to-state stable. arXiv:2507.02131, 2025.
- Devolder, Glineur & Nesterov. First-order methods of smooth convex optimization with inexact oracle. Math. Programming 146, 2014.
- Dohare et al. Loss of plasticity in deep continual learning. Nature 632, 2024.
- Han et al. FIRE: Frobenius-isometry reinitialization for balancing the stability-plasticity tradeoff. arXiv:2602.08040, 2026.
- He & Liu. Understanding and exploiting plasticity for non-stationary network resource adaptation. arXiv:2505.01584, 2025.
- Hernandez-Garcia, Dohare, Luo & Sutton. Reinitializing weights vs units for maintaining plasticity in neural networks. arXiv:2508.00212, 2025.
- Hespanha & Morse. Stability of switched systems with average dwell-time. IEEE CDC 1999.
- Jiang, Teel & Praly. Small-gain theorem for ISS systems and applications. Math. Control Signals Systems 7, 1994.
- Jiang & Wang. Input-to-state stability for discrete-time nonlinear systems. Automatica 37, 2001.
- Kumar, Marklund & Van Roy. Maintaining plasticity in continual learning via regenerative regularization. CoLLAs 2024.
- Liberzon. Switching in Systems and Control. Birkhäuser, 2003.
- Liu et al. Learning efficient convolutional networks through network slimming. ICCV 2017.
- Nikishin et al. Deep reinforcement learning with plasticity injection. NeurIPS 2023.
- Sokar, Agarwal, Castro & Evci. The dormant neuron phenomenon in deep reinforcement learning. ICML 2023.
- Sontag. Smooth stabilization implies coprime factorization. IEEE Trans. Automatic Control 34, 1989.
- Yu et al. Gradient surgery for multi-task learning. NeurIPS 2020.